

# Podstawy Uczenia Maszynowego

# Czym się zajmiemy na ćwiczeniach?

~~Ćwiczenia 1: metody liniowe~~

~~Ćwiczenia 2: metody oparte na sąsiedztwie~~

~~Ćwiczenia 3: metody jądrowe~~

**Ćwiczenia 4: metody probabilistyczne**

Ćwiczenia 5: klasyfikacja niezbalansowana

Ćwiczenia 6: selekcja cech i redukcja wymiarowości

Ćwiczenia 7: klasteryzacja

# Metody probabilistyczne

# Modele prawdopodobieństwa

# Prawdopodobieństwo a klasyfikacja

- dane definiują pewien **rozkład prawdopodobieństwa**:
  - $X$  ma **rozkład łączny (joint probability distribution)** rozkładów cech
  - $Y$  w klasyfikacji ma **rozkład kategoriyczny (categorical distribution)**
- formalnie mamy problem postaci:  $X \times Y \rightarrow [0, 1, \dots, K]$
- w praktyce chcemy zamodelować tę zależność za pomocą **funkcji**, mapującej cechy na klasy, z parametrami  $\theta$

$$f_{\theta}(x) : x \rightarrow [0, 1, \dots, K]$$

# Modele dyskryminatywne

- **modele dyskryminatywne (discriminative)** wprost rozwiązują zadanie klasyfikacji
- np. SVM, regresja logistyczna
- modelują one **warunkowy rozkład prawdopodobieństwa (conditional probability distribution)**:

$$P_{\theta}(Y|X = x) \quad P_{\theta}(y|x) : X \times Y \rightarrow [0, 1]$$

- to modelowanie może być:
  - explicite, z uczeniem np. przez MLE (regresja logistyczna)
  - implicite, bez rozkładu prawdopodobieństwa (SVM)

# Modele generatywne

- **modele generatywne (generative)** modelują proces, zgodnie z którym powstały dane
- modelują one **łączny rozkład prawdopodobieństwa (joint probability distribution):**

$$P_{\theta}(X, Y) \quad P_{\theta}(x, y) : X \times Y \rightarrow [0, 1]$$

- przykładowo:
  - Naive Bayes
  - Linear Discriminant Analysis (LDA)
  - Quadratic Discriminant Analysis (QDA)
- **ale:** jak tym klasyfikować?

# Klasyfikacja modelami generatywnymi

- metoda **maximum a posteriori (MAP)**
- wybieramy klasę o **najwyższym prawdopodobieństwie**
- z **twierdzenia Bayesa (Bayes theorem)**:

$$\arg \max_y P(y|x) = \arg \max_y \frac{P(x, y)}{P(x)} = \arg \max_y \frac{P(x|y)P(y)}{P(x)}$$

- $\arg \max(y)$  nie zależy od  $P(x)$ :

$$\arg \max_y P(y|x) = \arg \max_y P(x|y)P(y)$$



# Statystyka bayesowska

$$P(y|x) = \frac{P(x, y)}{P(x)} = \frac{P(x|y)P(y)}{P(x)}$$

| Element  | Nazwa                           | Opis                                                        |
|----------|---------------------------------|-------------------------------------------------------------|
| $P(y)$   | Praw. a priori (prior)          | Prawd. klasy $y$ zakładane z góry, przed stworzeniem modelu |
| $P(x y)$ | Wiarygodność (likelihood)       | Prawd. otrzymania danych $x$ , przy założeniu klasy $y$     |
| $P(x)$   | Dowód (evidence)                | Czynnik normalizujący (żeby prawd. sumowały się do 1)       |
| $P(y x)$ | Prawd. a posteriori (posterior) | Finalne prawd. klasy $y$ , otrzymane na podstawie danych    |

# Modele dyskryminatywne vs generatywne

## Modele dyskryminatywne:

- modelują rozkład warunkowy:  
 $P_{\theta}(y|x) : X \times Y \rightarrow [0, 1]$
- typowo lepsze wyniki w klasyfikacji
- potrafią tylko odróżniać klasy od siebie

## Modele generatywne:

- modelują rozkład łączny:  
 $P_{\theta}(x, y) : X \times Y \rightarrow [0, 1]$
- typowo gorsze wyniki w klasyfikacji - ale z ważnymi wyjątkami!
- potrafią:
  - klasyfikować
  - generować nowe próbki
  - imputować wartości brakujące
  - wykrywać outliery

# Regresja logistyczna

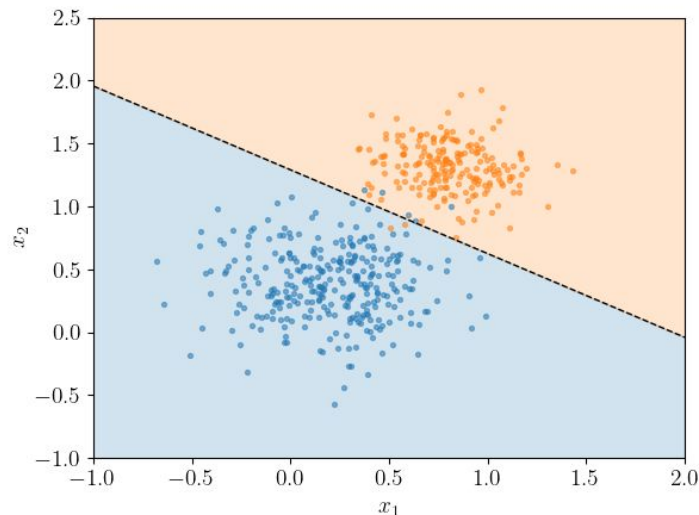
# Regresja logistyczna

- **idea:** dodać do regresji liniowej przekształcenie zwracające prawdopodobieństwa

- model postaci:

$$y = \sigma(x^T w) = \frac{1}{1 + e^{-x^T w}}$$

- **klasyfikator liniowy**, model dyskryminatywny
- wysoki bias, niska wariancja
- **funkcja kosztu:** entropia krzyżowa (cross-entropy)
- **trening:** Maximum Likelihood Estimation (MLE)



# Regresja logistyczna - zastosowania

- **baseline**
- **zastosowania real-time**
- **nauki przyrodnicze** - analiza regresji z binarną (binomial) zmienną zależną
- **nauki medyczne** - testy medyczne, np. Trauma and Injury Severity Score (TRISS)
- **predykcja prawdopodobieństwa** - reg. logistyczna bardzo precyzyjnie modeluje prawdopodobieństwo
- **sieci neuronowe** - jako ostatnia warstwa sieci

# Model regresji logistycznej

- model to przekształcenie regresji liniowej z użyciem **funkcji logistycznej (sigmoidy)**:

$$y = \sigma(x^T w) = \frac{1}{1 + e^{-x^T w}}$$

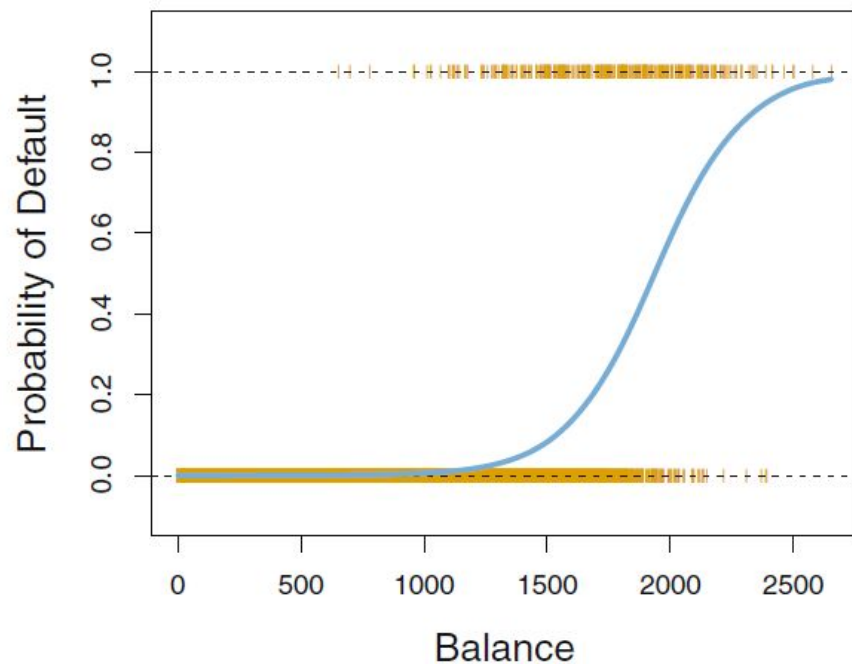
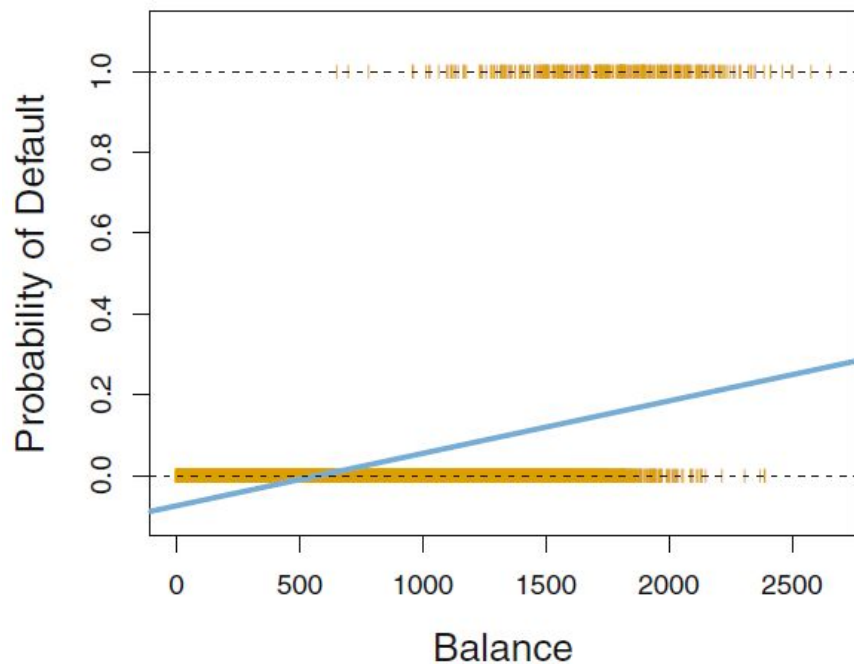
- zamienia zakres wartości z  $(-\infty, +\infty)$  na  $[0, 1]$ , czyli mamy **prawdopodobieństwo klasy pozytywnej**

$$y = p(x) = P(y = 1|x)$$

- model zakłada, że regresja liniowa przewiduje **logarytm szans (log-odds)**:

$$x^T w = \ln \frac{p(x)}{1 - p(x)}$$

# Regresja liniowa vs logistyczna



# Regresja logistyczna - obliczanie

- **nie ma wzoru analitycznego** - trzeba wyprowadzić funkcję kosztu i minimalizować
- metoda największej wiarygodności - **maximum likelihood estimation, MLE**
- **funkcja wiarygodności (likelihood function)** to prawdopodobieństwo wygenerowania zbioru danych przez model:

$$\mathcal{L}(w) = \prod_{i:y_i=1} p(x_i) \prod_{i:y_i=0} (1 - p(x_i))$$

- obliczamy na zbiorze treningowym,  $p(x)$  to predykcja modelu
- **wysoka, gdy:** wysokie prawd. dla klasy pozytywnej i niskie dla negatywnej



# Problemy z likelihood function

$$\mathcal{L}(w) = \prod_{i:y_i=1} p(x_i) \prod_{i:y_i=0} (1 - p(x_i))$$

- **stabilność numeryczna:**

- **nigdy** nie mnożymy małych liczb w pętli!
- tu są z zakresu  $[0,1]$ , i to dla całego zbioru treningowego

- **maksymalizacja:**

- wszystkie typowe optymalizatory minimalizują

# Wyprowadzenie MLE

$$\mathcal{L}(w) = \prod_{i:y_i=1} p(x_i) \prod_{i:y_i=0} (1 - p(x_i))$$

$\ln b + \ln c = \ln (b * c)$   
 $a \cdot \ln(b) = \ln(b^a)$

$$\mathcal{L}(w) = \prod_{i=1}^n p(x_i)^{y_i} \cdot (1 - p(x_i))^{1-y_i}$$

# łączymy za pomocą klasy (0 lub 1)

$$\ln \mathcal{L}(w) = \sum_{i=1}^n \ln (p(x_i)^{y_i} \cdot (1 - p(x_i))^{1-y_i})$$

# log iloczynu = suma log

$$\ln \mathcal{L}(w) = \sum_{i=1}^n \ln (p(x_i)^{y_i}) + \ln (1 - p(x_i))^{1-y_i})$$

# log iloczynu = suma log

$$\ln \mathcal{L}(w) = \sum_{i=1}^n y_i \ln (p(x_i)) + (1 - y_i) \ln (1 - p(x_i))$$

# wyciągamy potęgę z log

# Wyprowadzenie MLE

$$\ln \mathcal{L}(w) = \sum_{i=1}^n y_i \ln(p(x_i)) + (1 - y_i) \ln(1 - p(x_i))$$

$$NLL(w) = - \sum_{i=1}^n y_i \ln(p(x_i)) + (1 - y_i) \ln(1 - p(x_i)) \quad \# \text{ mnożymy przez } -1$$

- **negative log-likelihood (NLL)** - nasza funkcja kosztu do minimalizacji
- w tym szczególnym przypadku jest równa innej ciekawej funkcji

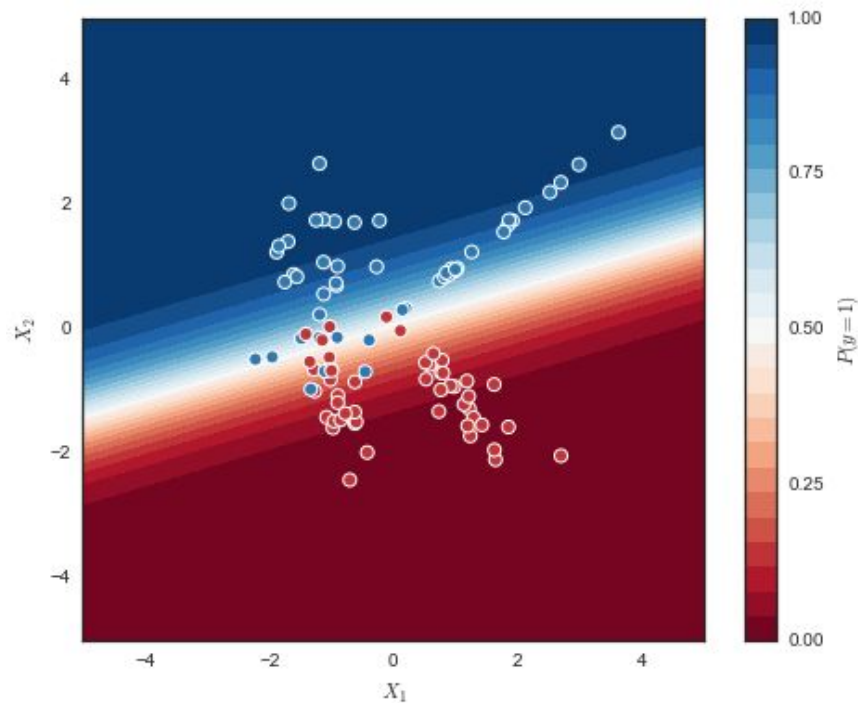
# Binary cross-entropy (BCE)

$$BCE(w) = - \sum_{i=1}^n y_i \ln(p(x_i)) + (1 - y_i) \ln(1 - p(x_i))$$

- **binarna entropia krzyżowa (binary cross-entropy)**, nazywana też **log-loss**
- mierzy różnicę dwóch rozkładów prawdopodobieństwa
- **własności:**
  - różniczkowalna
  - wypukła
  - zero dla liniowo separowalnych klas - wtedy **nieskończenie wiele** rozwiązań

# Granica decyzyjna a prawdopodobieństwa

- **pewność (score)** - odległość od granicy decyzyjnej
- wprost tłumaczona na prawdopodobieństwo
- SVM to ignorować, regresja logistyczna używa



# Regresja logistyczna - wiele klas

- klasyczna wersja jest do **klasyfikacji binarnej**, dla wielu klas mamy dwie możliwości
- **softmax:**
  - multinomial loss
  - softmax + cross-entropy (zamiast sigmoidy i BCE)
  - dalej różnica między rozkładami prawdopodobieństwa
  - poprawniejsza matematycznie wersja
- **One-vs-Rest (OvR):**
  - trenujemy  $K$  modeli (dla  $K$  klas)
  - każdy uczy się oddzielać "swoją" klasę od  $K-1$  pozostałych
  - zwykle tworzy problemy (mocno) niezbalansowane

# Regresja logistyczna - regularyzacja

$$L(w) = BCE(w) + \alpha ||w||_1 + \lambda ||w||_2^2$$

- L1, L2, ElasticNet - co tylko chcemy, są do tego solwery
- L2 jest najszybsze
- **wymaga standaryzacji zmiennych**

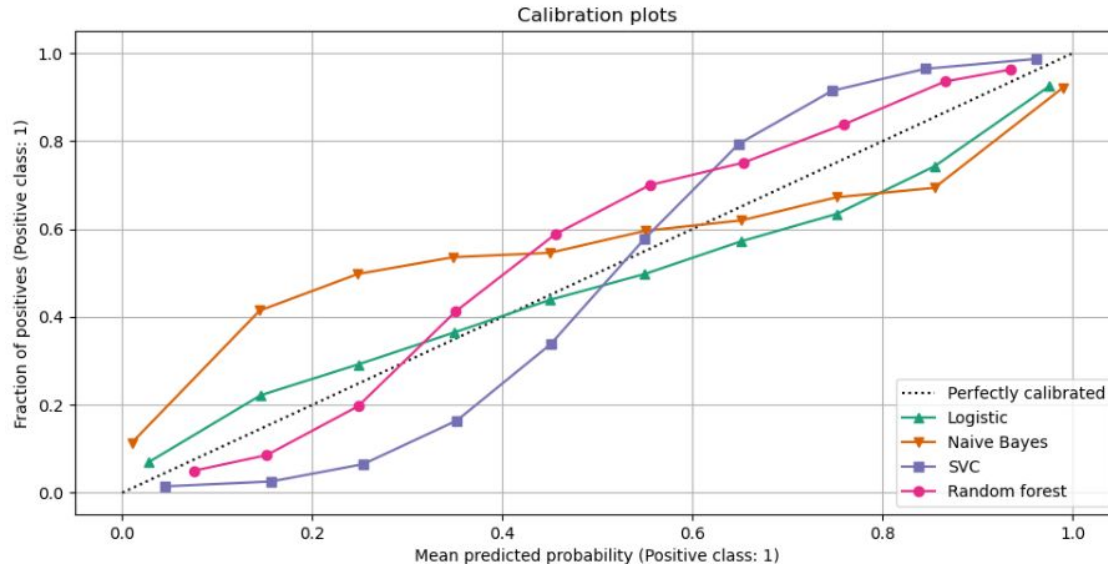
# Regresja logistyczna - obliczanie

- **dużo solverów**, typowo bardzo szybkie i skalowalne
- solwery ([tabela ze Scikit-learn](#)) zależą od regularyzacji i danych
- rekomendacje:
  - reg. L2: L-BFGS (default)
  - reg. L1 i ElasticNet: SAGA
  - dane rzadkie: Newton conjugate gradients
- te 3 wspierają wszystkie softmax (multinomial loss)
- bardzo szybkie i skalowalne pod kątem liczby próbek oraz liczby cech



# Model probabilistyczny

- regresja logistyczna jest **dobrze skalibrowana (well calibrated)** - precyzyjnie oblicza prawdopodobieństwa
- wynika z faktu, że wprost optymalizuje entropię krzyżową
- można ją "dołożyć" do innego klasyfikatora, żeby ulepszyć jego estymaty - dokładnie to robi się z SVM



# Kalibracja prawdopodobieństwa - materiały

- [documentacja scikit-learn](#) bardzo dobrze to opisuje
- NannyML - [Bad Machine Learning models can still be well-calibrated](#)
- Arize.AI - [Calibration Curves: What You Need To Know](#)
- Ploomber - [Can I trust my model's probabilities? A deep dive into probability calibration](#)
- Neptune.AI - [Brier Score: Understanding Model Calibration](#)
- [Expected Calibration Error \(ECE\): A Step-by-Step Visual Explanation](#)

# Regresja logistyczna - zalety i wady

## Zalety:

- proste
- szybkie i skalowalne
- dobre predykcje prawdopodobieństw
- mało hiperparametrów, łatwo tune'ować
- interpretowalność

## Wady:

- mała pojemność modelu
- nie bierze za bardzo pod uwagę interakcji cech
- czuły na dobór hiperparametrów

# Klasyfikator Bayesa

# Klasyfikator Bayesa

- **klasyfikator Bayesa (Bayes classifier)** wprost stosuje definicję:

$$f(x) = \arg \max_y P(y|x) = \arg \max_y P(x, y) = \arg \max_y P(x|y)P(y)$$

- prior  $P(y)$  - rozkład klas w zbiorze treningowym
- wiarygodność  $P(x|y)$  - sprawdzamy wszystkie możliwe wartości  $x$  dla każdej z klas  $y$
- **to nie jest wykonalne!**

# Klasyfikator Bayesa w teorii ML

- klasyfikator Bayesa to **optymalny klasyfikator (optimal classifier)**
- modeluje idealnie procedurę generującą dane, więc ma całkowitą wiedzę
- popełnia najmniejszy możliwy błąd - **błąd Bayesa (Bayes error rate, BER)**
- używane w analizie teoretycznej ML, na danych syntetycznych

# Czy da się coś z tym zrobić?

- **idea:** założmy, że dane pochodzą z jakiegoś konkretnego rozkładu
- np. jak wszystkie cechy są binarne, to założmy rozkład dwumianowy (binomial)
- rozkład jest d-wymiarowy (tyle, co cech)
- **zalety:**
  - daje zamknięty wzór na wiarygodność i skończoną liczbę parametrów
- **wada - klątwa wymiaru:**
  - liczba parametrów rośnie liniowo z wymiarem
  - trzeba eksponencjalnie dużo punktów, żeby wyestymować parametry

# Naiwny klasyfikator Bayesa



# Naiwny klasyfikator Bayesa

- **naive Bayes classifier**
- **idea:** założmy, że cechy są warunkowo niezależne, w obrębie każdej klasy
- pozwala rozłożyć  $d$ -wymiarowy rozkład na iloczyn  $d$  1-wymiarowych rozkładów
- **zalety:** trywialnie proste i szybkie
- **wady:** bardzo mocne i nierealne założenie, pogarszające wyniki

# Naive Bayes - wyprowadzenie

- w rozkładzie łącznym, każda cecha zależy od wszystkich pozostałych:

$$P(x, y_k) = P(x_1|x_2, \dots, x_d, y_k) * P(x_2|x_3, \dots, x_d, y_k) \dots * P(x_d|y_k) * P(y_k)$$

- **założenie warunkowej niezależności (conditional independence assumption)** mówi, że w obrębie klasy uznajemy cechy za niezależne:

$$P(x_i|x_{i+1}, \dots, x_d, y_k) = P(x_i|y_k)$$

- rozkład łączny redukuje się do iloczynu rozkładów dla cech:

$$P(x_1, \dots, x_d, y_k) = P(y_k) \prod_{i=1}^d P(x_i|y_k)$$

# Naive Bayes - trening

- dla każdej z  $K$  klas mamy  $d$  1-wymiarowych rozkładów, ale dalej trzeba oszacować ich parametry

$$P(x_1, \dots, x_d, y_k) = P(y_k) \prod_{i=1}^d P(x_i | y_k)$$

- problem:** mnożymy prawdopodobieństwa -> underflow!
- rozwiązanie:** obliczyć eksponentę i logarytm - iloczyn zamieni się na sumę logarytmów

# Naive Bayes - log-sum-exp trick

- ta operacja to **log-sum-exp trick**

$$P(x, y_k) = P(y_k) \prod_{i=1}^d P(x_i | y_k) = \exp \left( \log P(y_k) + \sum_{i=1}^d \log P(x_i | y_k) \right)$$

- jeżeli nie chcemy prawdopodobieństw, tylko klasyfikację, to można opuścić eksponentę

$$\arg \max_y P(x, y) = \arg \max_y \left( \log P(y) + \sum_{i=1}^d \log P(x_i | y) \right)$$

# Naive Bayes - trening

- mamy  $d$  1-wymiarowych rozkładów, ale dalej trzeba oszacować ich parametry

$$\arg \max_y P(x, y) = \arg \max_y \left( \log P(y) + \sum_{i=1}^d \log P(x_i|y) \right)$$

- **pytanie:** jakie to w ogóle są rozkłady?

# Naive Bayes - typy

- **wybieramy sobie rozkład**
- daje to różne wersje Naive Bayesa, typowe to:
  - categorical NB
  - binomial (Bernoulli) NB
  - multinomial NB
  - Gaussian NB

# Categorical Naive Bayes

- "typowy" NB ze statystyki
- **rozkład:** kategoriyczny (categorical)
- **dla cech:** kategoriycznych (nieuporządkowanych zbiorów kategorii)
- **parametry:** ile razy dana kategoria wystąpiła w danej klasie
- **wiarygodność dla cechy:**

$$P(x_i = t | y = c; \alpha) = \frac{N_{tic} + \alpha}{N_c + \alpha * n_i}$$

$N_{(tic)}$  - ile razy wartość "t" wystąpiła dla i-tej cechy w klasie "c"

$N_c$  - łączna liczba próbek w klasie "c"

$n_i$  - liczba kategorii (unikalnych wartości) dla i-tej cechy

- $\alpha$  - tzw. **Laplace smoothing**, unika dzielenia przez zero

# Binomial Naive Bayes

- **rozkład:** dwumianowy (binomial)
- **dla cech:** binarnych
- **parametry:** liczba wierszy o wartości cechy 1 (w obrębie klasy)
- **wiarygodność dla cechy:**

$$P_{binomial}(x_i|y_k) = P(x_i = 1|y_k)^{x_i} * P(x_i = 0|y_k)^{(1-x_i)}$$

- explicite modeluje brak danej cechy



# Multinomial Naive Bayes

- **rozkład:** wielomianowy (multinomial)
- **dla cech:** liczb całkowitych, np. zliczenia (counts)
- **parametry:** proporcja danej cechy (w obrębie klasy)

$$\theta_{yi} = \frac{N_{yi} + \alpha}{N_y + \alpha}$$

$N_{yi}$  - suma i-tej cechy w klasie y  
 $N_y$  - suma wszystkich cech w klasie y

- **wiarygodność dla cechy:** nie oblicza się, bo jest fajniejszy wzór :)

# Multinomial Naive Bayes

- dzięki log-sum-exp trick, cały wzór na predykcję na multinomial NB redukuje się do:

$$f_{multinomial}(x) = \arg \max_y \left( \log P(y) + \sum_{i=1}^d x_i \log \theta_{yi} \right) \quad \theta_{yi} = \frac{N_{yi}}{N_y}$$

$x_i$  - wartość i-tej cechy dla nowego przykładu

- **obserwacja:** to zwykły klasyfikator liniowy

$$f_{multinomial}(x) = \arg \max_y (b_y + w_y^T x)$$

# Gaussian Naive Bayes

- **rozkład:** normalny / Gaussowski (normal / Gaussian)
- **dla cech:** rzeczywistych
- **parametry:**  $\mu$ ,  $\sigma^2$  - średnia i wariancja cechy (w obrębie klasy)
- **wiarygodność dla cechy:**

$$P_{Gaussian}(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_{yi}^2}} \exp\left(-\frac{(x_i - \mu_{yi})^2}{2\sigma_{yi}^2}\right)$$

- zakłada, że cechy ciągłe mają rozkłady normalne w obrębie klasy

# Gaussian Naive Bayes

- oczywiście też korzystamy z log-sum-exp trick
- wyprowadzenie log-likelihood dla zainteresowanych :)
- wzór:

$$\log P_{Gaussian}(x_i|y) = -\frac{1}{2} \ln(2\pi\sigma_{yi}^2) - \frac{(x_i - \mu_{yi})^2}{2\sigma_{yi}^2}$$

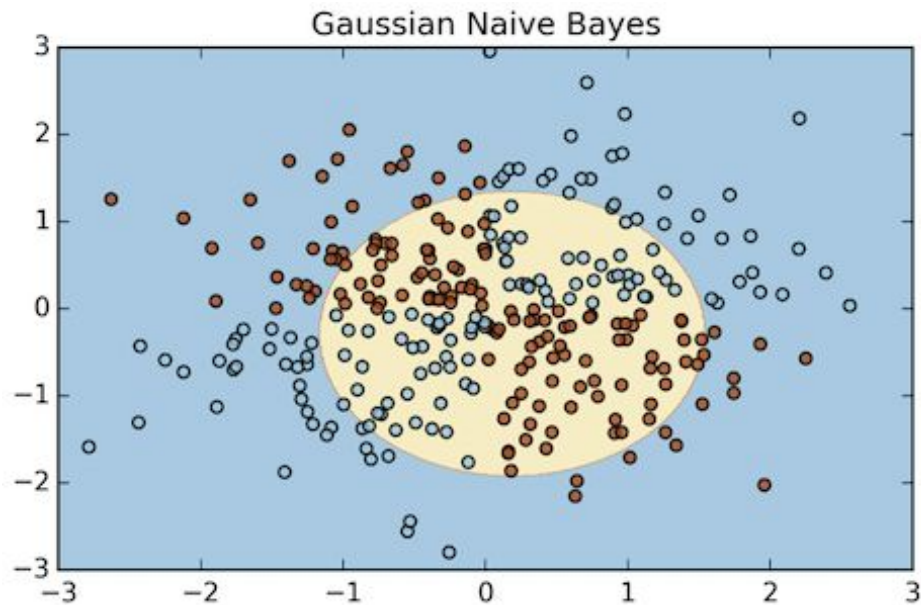
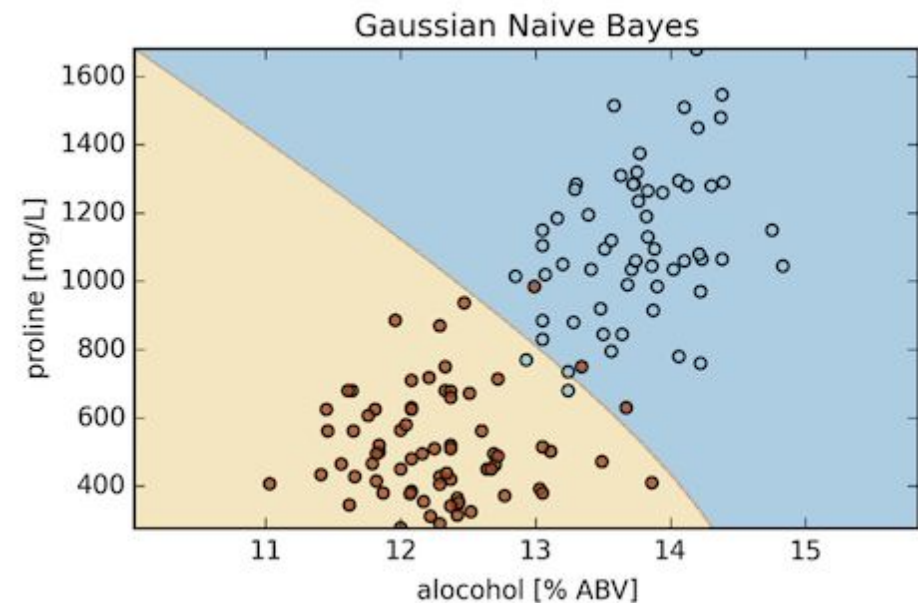
# Naive Bayes - trening

- trening NB = obliczenie parametrów na zbiorze treningowym
- realizuje **maximum likelihood estimation (MLE)**
- mamy trening MLE i predykcję MAP

# Naive Bayes - granice decyzyjne

- czy Naive Bayes jest liniowym, czy nieliniowym klasyfikatorem?
- **to zależy od typu:**
  - binomial, multinomial - liniowy
  - Gaussian - nieliniowy
- w przypadku Gaussian NB granica decyzyjna to funkcja **piecewise quadratic**
- **uwaga:** niektórzy błędnie uznają, że Gaussian NB jest liniowy, ale mylą go z Linear Discriminant Analysis (LDA) - [wyjaśnienie](#)

# Gaussian NB - granice decyzyjne



# Naive Bayes - predykcja

- predykcja naiwnego klasyfikatora Bayesa to:

$$\arg \max_y P(x, y) = \arg \max_y \left( \log P(y) + \sum_{i=1}^d \log P(x_i|y) \right)$$

- działa dla dowolnej liczby klas
- **predykcje prawdopodobieństw:** liczymy powyższy wzór dla każdej klasy, liczymy eksponentę i dzielimy przez sumę, żeby był rozkład prawd.
- **ale:** daje bardzo kiepskie predykcje prawdopodobieństw



# Naive Bayes - złożoność obliczeniowa

- trening to przejście 1 raz po danych i estymacja parametrów (mamy wzory analityczne!)
- predykcja to zaaplikowanie wzorów dla każdego z wymiarów
- **złożoność treningu:**  $O(nd)$
- **złożoność predykcji:**  $O(kd)$

# Naive Bayes - hiperparametry

- **brak**
- i zaleta, i wada
- jedyny sposób na poprawę wyników to lepszy feature engineering

# Naive Bayes - zastosowania

- **genetyka:**
  - dla cech z sekwencjonowania genów  $d \gg n$ , a cechy są dość niezależne
  - naive Bayes wymaga mało danych
- **prosta klasyfikacja tekstów:**
  - tam, gdzie temat tekstu ma większe znaczenie, a dokładna semantyka mało
  - zliczamy wystąpienia poszczególnych słów - multinomial NB
  - założenie warunkowej niezależności całkiem dobrze spełnione
  - np. detekcja spamu, klasyfikacja tematu

# Naiwny klasyfikator Bayesa - zalety i wady

## Zalety:

- prostota
- bardzo szybki trening i predykcja
- skalowalność
- działa dobrze dla małych zbiorów
- działa dobrze dla  $d \gg n$  (bardzo odporny na klątwę wymiarowości)
- dobre wyniki, kiedy są (w miarę) spełnione założenia

## Wady:

- słabe wyniki dla dużych zbiorów z cechami ciągłymi
- nie zyskuje zbyt wiele dla większych danych

**Slajdy bonusowe**

# Kernel Density Estimation (KDE)

# Naive Bayes i cechy ciągłe

- Gaussian NB zakłada, że cechy ciągłe mają rozkład normalny, i to w obrębie każdej klasy
- **problem:** to bardzo mocne założenie!
- **obserwacja:** mamy dane dla każdej cechy, mają jakiś rozkład
- **idea:** po co w ogóle zakładać rozkład, jak możemy go obliczyć wprost z danych?

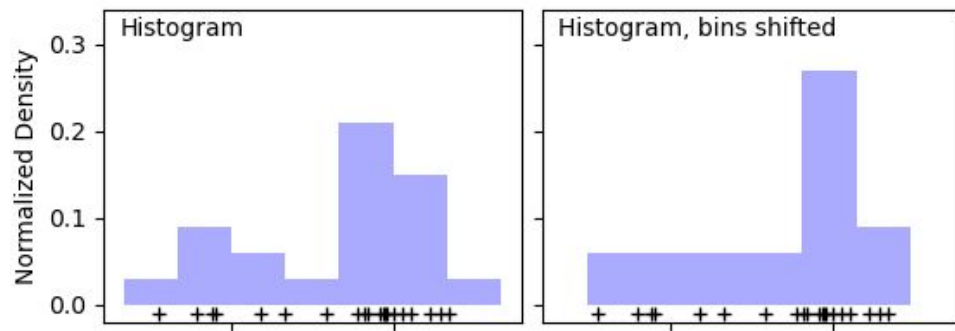
# Kernel Density Estimation (KDE)

- cecha ciągła to formalnie **funkcja gęstości prawdopodobieństwa (probability density function, PDF)**
- empiryczne oszacowanie funkcji na podstawie danych realizuje **kernel density estimation (KDE)**
- inne nazwy: Parzen windows, Parzen estimator
- wartości są dyskretne, ale to tylko próbki z jakiegoś rozkładu, który jest ciągły!
- kernele pozwalają "wygładzić" rozkład na podstawie próbek



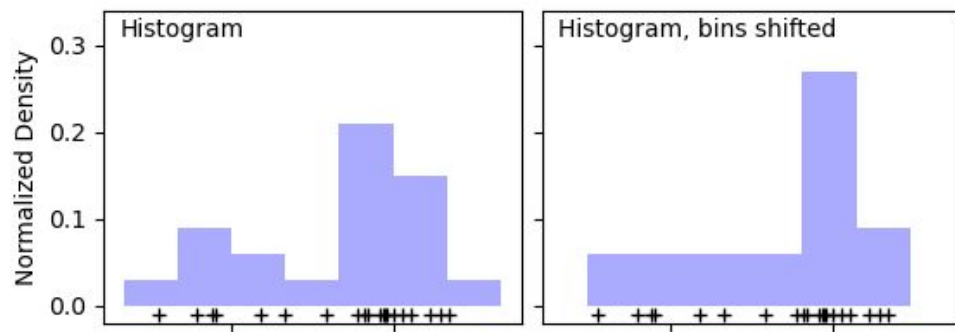
# Histogram jako KDE

- histogram to najprostsza reprezentacja PDF
- ale ma szereg problemów
- dyskretny, z podziałem na kubetki (bins) - prawdziwe PDF są ciągłe
- jak dobrać szerokość i liczbę kubetków?
- w zależności od doboru kubetków dostajemy zupełnie inny obraz sytuacji



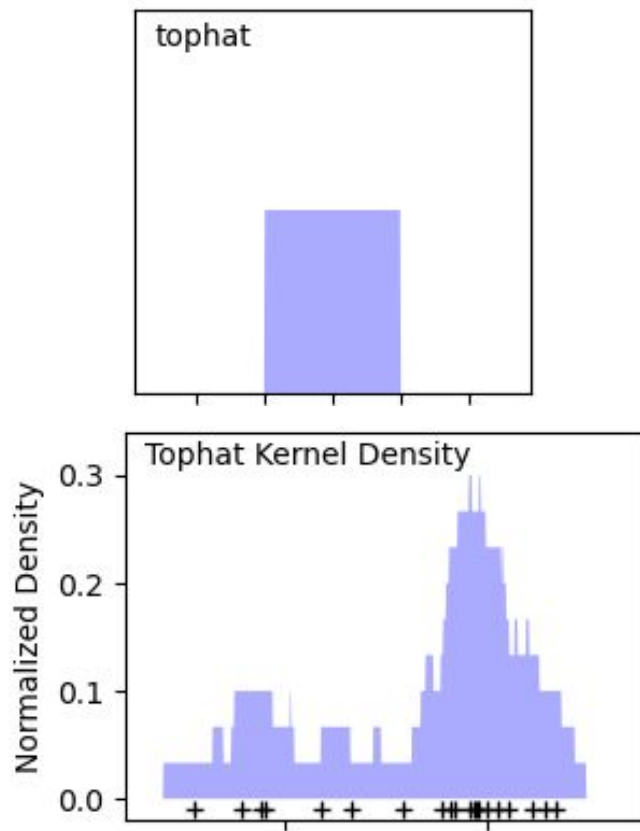
# Histogram jako Tetris

- każda wartość cechy to punkt na osi
- tradycyjny histogram = dzielimy oś na kubeczki o jednakowej szerokości
- każda wartość wrzuca klocek o wysokości 1 do swojego kubeczka
- normalizujemy, żeby sumowało się do 1
- **idea:** wyrzucmy kubeczki i po prostu rzucamy klocki na punkty



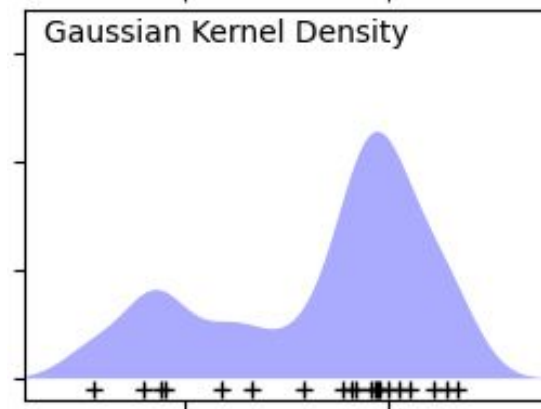
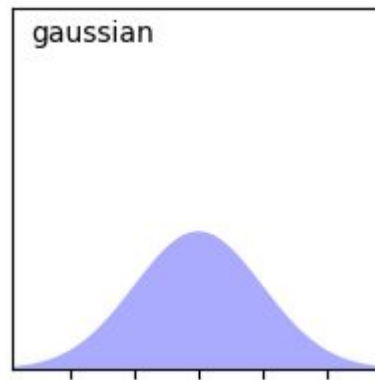
# Tophat kernel

- założmy, że zrzucamy prostokątne klocki, gdzie środek wypada na punkcie
- jest to **tophat kernel**
- kernel = funkcja podobieństwa
- jak sąsiedzi są podobni, to podobieństwo to 1 (wysokość klocka), w przeciwnym razie 0
- granicę wyznacza szerokość klocka - **szerokość (bandwidth)** kernela



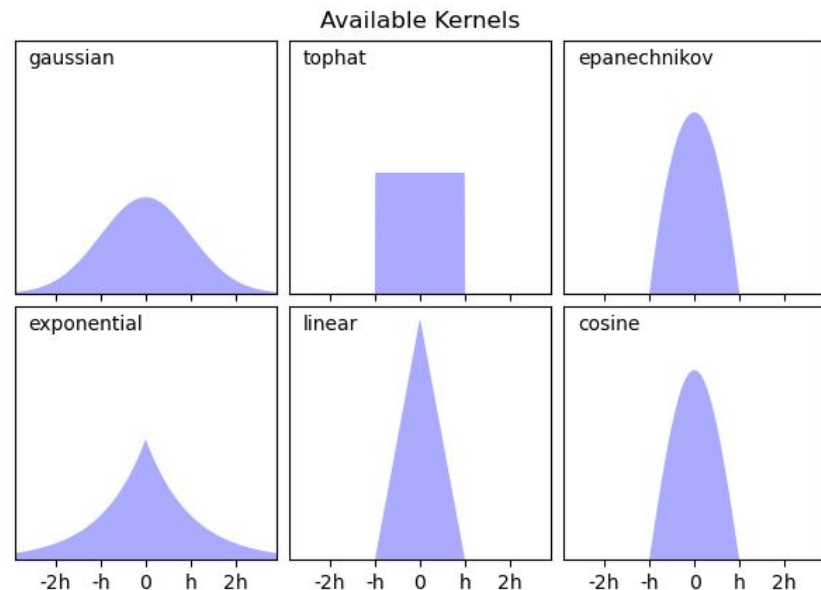
# Gaussian kernel

- założmy, że nasze klocki są krzywą dzwonową
- jest to **Gaussian kernel**
- mamy nasz gładki rozkład prawdopodobieństwa!
- kształt krzywej wyznacza bandwidth - im mniejszy, tym "ostrzejszy" dzwon

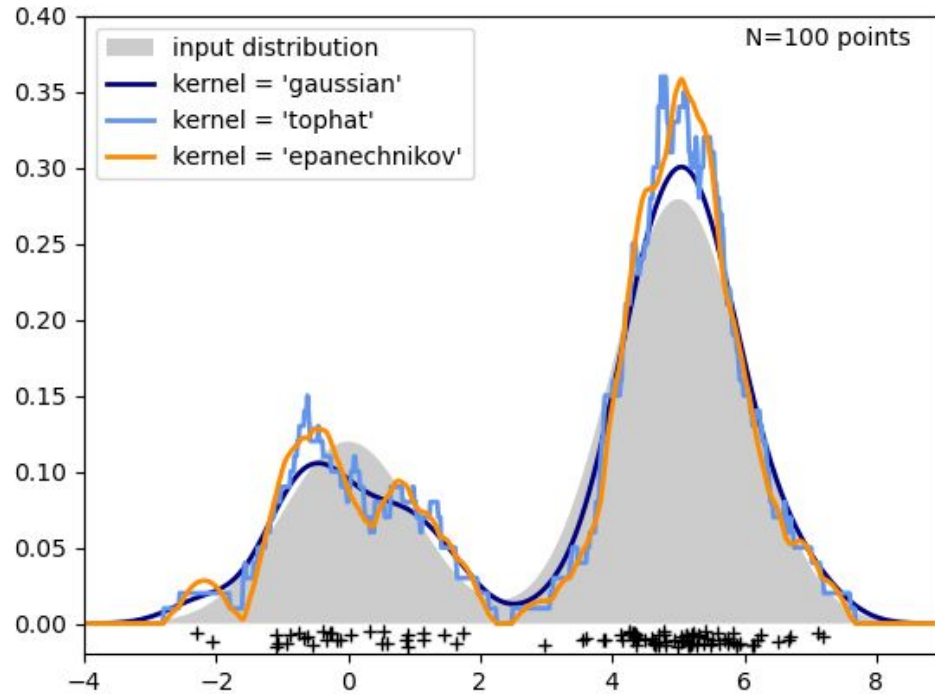


# Kernele w KDE

- kernel = funkcja podobieństwa
- **symetryczny**
- **hiperparametr**: szerokość (bandwidth)
- najczęściej używany jest Gaussowski
- niektóre mają **skończony support** (biorą pod uwagę tylko bandwidth)



# Kernele w KDE - wizualizacja



# Po co nam KDE?

- KDE daje nam normalny rozkład prawdopodobieństwa
- możemy **próbować** z takiego rozkładu, np. imputować wartości brakujące
- możemy **obliczać wiarygodność** dla nowego punktu
- zamknięty wzór nie istnieje, ale mamy nasze empiryczne PDF
- **obserwacja:**
  - wzór nie istnieje - wszystko jest empiryczne
  - opieramy się na pozostałych punktach
- czy coś nam to przypomina?

# KDE a najbliżsi sąsiedzi

- KDE jest bardzo podobne do kNN!
- **KDE jest leniwe:**
  - podczas treningu zapamiętujemy zbiór
  - podczas predykcji sumujemy kernel dla nowego punktu i najbliższych sąsiadów
- **KDE jest nieparametryczne:**
  - nie uczymy się żadnych parametrów
  - zapamiętujemy tylko zbiór



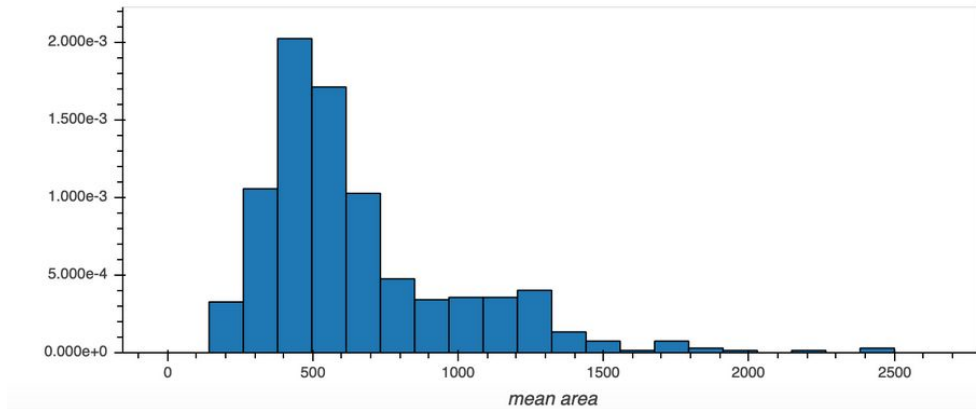
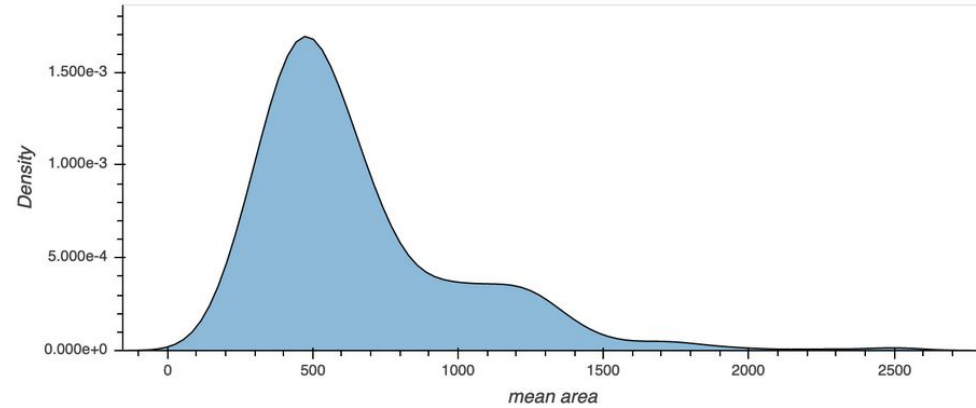
# KDE - złożoność obliczeniowa

- trening - zbudowanie struktury do wyszukiwania najbliższych sąsiadów
- obliczenie wiarygodności - znalezienie najbliższych sąsiadów i zsumowanie ich kerneli
- typowo używa **ball tree** - podobne do k-d tree, ale grupuje w "kule" najbliższych sąsiadów
- **złożoność treningu:**  $O(n \cdot \log(n))$
- **złożoność obliczania wiarygodności:**  $O(\alpha \cdot \log(n))$
- $\alpha$  to jakaś stała, ale ciężko powiedzieć jaka - mamy znaleźć wszystkie dość bliskie punkty (do zadanej dokładności)

# KDE - zastosowania

- **wizualizacja:**
  - ładniejsze, "wygładzone" histogramy
  - realnie lepsze pojęcie o danych w wielu wypadkach, szczególnie w 2D
- **optymalizacja hiperparametrów:**
  - część grupy algorytmów Bayesian Hyperparameter Optimization (Bayesian HPO)
  - algorytm Tree Parzen Estimator (TPE)
  - biblioteka [Optuna](#)
- **KDE Naive Bayes**

# KDE vs histogram - wizualizacja



# KDE Naive Bayes

- KDE + Naive Bayes = bardzo elastyczny i mocny klasyfikator
- nazywany **Flexible Bayes** albo **KDE Naive Bayes**

John, George H., and Pat Langley. *"Estimating continuous distributions in Bayesian classifiers."* ([link](#))

- **rozkład łączny** dla każdej klasy estymujemy z KDE
- **usuwa** założenie warunkowej niezależności cech - już nie jest tak naiwnie!
- **dużo lepsze wyniki**, jeżeli cechy są dalekie od normalnych

# KDE Naive Bayes

- **zalety:**

- może uzyskać dużo lepsze wyniki dla danych ciągłych
- nie trzeba zakładać i sprawdzać rozkładów cech

- **wady:**

- dodatkowe hiperparametry: kernel, bandwidth
- dużo większa złożoność predykcji - algorytm leniwy

# KDE w optymalizacji hiperparametrów

- algorytm Tree Parzen Estimator (TPE) opiera się o KDE
- część grupy algorytmów Bayesian Hyperparameter Optimization (Bayesian HPO)
- **idea:**
  - regresja nad hiperparametrami modelu
  - hiperparametry = cechy w regresji, zmienna zależna = performance modelu
  - ewaluacja funkcji to trening modelu na danych hiperparametrach
  - chcemy znaleźć maksimum, czyli najlepszy zestaw hiperparametrów
  - robimy to krok po kroku, jako proces optymalizacji

# KDE w optymalizacji hiperparametrów

- TPE robi **interpolację** przestrzeni hiperparametrów z pomocą KDE
- pozwala efektywnie przeszukiwać wielowymiarową przestrzeń hiperparametrów
- [Automated Machine Learning - Tree Parzen Estimator \(TPE\)](#)
- [NeuPy - Tree-structured Parzen Estimators \(TPE\)](#)
- ["Multivariate" TPE Makes Optuna Even More Powerful](#)
- [CrossValidated - What is the "tree" structure in Tree Parzen Estimators?](#)
- ["Algorithms for Hyper-Parameter Optimization" J. Bergstra et al.](#)
- ["Tree-Structured Parzen Estimator: Understanding Its Algorithm Components and Their Roles for Better Empirical Performance" S. Watanabe](#)